

JB 준법 코파일럿 — 로직 설명서

에이전트 파이프라인 · 모듈별 알고리즘 · 점수 산식 · 폴백 전략

버전 0.1.0 (MVP) · 2026-06-11 · 함께 보기: 사용 설명서, 설계 문서

1 전체 파이프라인

무거운 에이전트 프레임워크 대신 **명시적 파이썬 파이프라인**(`app/pipeline/orchestrator.py`)으로 구성해 모든 단계가 디버깅·설명 가능합니다. ②③과 ④⑤는 각각 `asyncio.gather` 로 병렬 실행됩니다.

```
POST /api/review {text, content_type?, language, product_facts?}
| (8자 미만 → skipped=True 즉시 반환)
├ ① 콘텐츠 유형 분류 — classifier.py      예금/투자/대출 + 신뢰도
├ ② 규제 근거 검색(RAG) — retriever.py     코퍼스 top-k 조항 + 유사도
├ ③ 룰엔진(결정적) — rules_engine.py       위반 룰 + span + 수정안
├ ④ LLM 심의(뉴앙스) — llm_reviewer.py    ②근거+③결과 기반 지적·수정안
├ ⑤ 오인 시뮬레이터 — simulator.py        페르소나 역할극 + 오해 판정
├ ⑥ 리포트 종합 — report.py               준법점수·등급·하이라이트·인용
├ ⑦ 승인/반려 (별도 API) — store.py       Human-in-the-loop
└ ⑧ 감사로그 + 다국어 — store.py / multilingual.py
```

설계 원칙	구현
LLM은 보조, 판정의 빠대는 결정적	룰엔진 결과는 LLM 가용 여부와 무관하게 항상 포함. LLM은 뉴앙스 리스크를 "추가"만 함
허위 인용 금지	모든 인용은 검색된 코퍼스의 조항 id로 제한 (§5.2, §6.3)
전 기능 폴백	LLM 의존 모듈(④⑤, 번역)은 모두 룰 기반 결정적 폴백 보유 → 오프라인 데모·테스트 가능

2 ① 콘텐츠 유형 분류기 `app/pipeline/classifier.py`

유형별 키워드 사전(가중치 1~2)에 대한 정규식 매칭 횟수 합산으로 점수를 계산합니다.

```

score(유형) =  $\sum$  ( 가중치  $\times$  본문 내 패턴 매치 횟수 )
deposit      : 예금|적금|예치|만기|이자율|예금자보호( $\times 2$ )  $\cdot$  정기|입출금|세전|세후( $\times 1$ )  $\cdot$ 
savings|deposit( $\times 2$ )
investment   : 펀드|투자|수익률|운용|주식|채권|ETF( $\times 2$ )  $\cdot$  원금손실|위험등급( $\times 1$ )  $\cdot$  fund|invest( $\times 2$ )
loan         : 대출|융자|한도|상환|연체( $\times 2$ )  $\cdot$  승인|신용점수|중도상환( $\times 1$ )  $\cdot$  loan|credit( $\times 2$ )

content_type = argmax score,    confidence = score(best) /  $\sum$  score(전체)

```

- 전체 점수 합이 0이면(금융 키워드 없음) `deposit` 기본값 + 신뢰도 0.0 반환.
- UI에서 유형을 수동 지정하면 분류 결과는 참고 정보로만 표시되고 심의는 지정 유형으로 라우팅됩니다.

3 ② 규제 근거 검색 (RAG) `app/pipeline/retriever.py`

3.1 코퍼스

`app/data/regulations.json` — 12개 조항. 각 항목은 `id / law / article / title / text / types(적용유형) / effective_date / source_url` 메타데이터를 가지며, 출처 인용과 버전 관리(규제 변경 추적)의 토대가 됩니다. 검색 대상은 `유형 일치 U common` 조항으로 필터링됩니다.

3.2 질의 확장

광고 문구는 짧아 법률 용어와 어휘가 겹치지 않으므로, 본문에서 위험 신호를 감지해 법률 도메인 시드 키워드를 질의에 덧붙입니다.

"확정 무조건 절대 100% guaranteed"	→ + "단정적 판단 보장 확실 오인"
"원금 보장 principal risk"	→ + "원금보장 손실보전 이익보장 예금자보호 한도"
"% 금리 이자 수익률 rate"	→ + "이자율 수익률 세전 세후 우대조건 변동"
"최고 최저 1위 유일"	→ + "최상급 거짓 과장 비교 표시 광고" (등 6종)

3.3 스코어링 (이중 백엔드)

```

# 기본: 어휘 검색 (의존성 0, 결정적)
score(d) = |q n d| / (  $\sqrt{|d|} \cdot \sqrt{|q|}$  )      # 정규화 토큰 중첩 (코사인 유사 형태)

# 업그레이드: sentence-transformers 설치 시 자동 전환
score(d) = BGE-m3(query)  $\cdot$  BGE-m3(doc)      # 정규화 임베딩 내적 (코사인)

```

- top-k = 4, 점수 임계값 0.05 미만은 버림 → 결과가 0건이면 `sufficient=false` 로 표시하고 UI에 "근거 불충분 — 인용 생략"을 노출합니다 (허위 인용 방지).
- 임베딩 백엔드는 try-import + 실패 시 `None` 반환으로 어휘 검색에 자동 폴백합니다.

3.4 검색 세팅값 — 값·근거

세팅	값	근거
top-k	4	LLM 컨텍스트에 주입할 조항 수를 4개로 제한 → 무관 조항 노이즈·토큰 낭비를 막고 핵심 근거만 전달. 인용 화이트리스트도 이 4개로 좁혀져 환각 인용 표면적을 줄임.
점수 임계값	0.05	이 미만은 사실상 무관 → 버려서 허위 인용 방지. 결과 0건이면 <code>sufficient=false</code> 로 "근거 불충분 — 인용 생략" 표기.
기본 백엔드	어휘검 색	의존성 0·결정적·즉시 구동(하드마감 내 1-커맨드 실행/테스트 우선). <code>sentence-transformers</code> 설치 시 BGE-m3 임베딩 코사인으로 자동 전환되도록 인터페이스 분리 → 정확도 업그레이드 경로 확보.

4 ③ 룰엔진 `app/pipeline/rules_engine.py` · `app/data/rules.yaml`

규제에서 파생한 결정적 체크. 룰은 YAML로 외부화되어 코드 수정 없이 추가·수정할 수 있습니다.

4.1 룰 스키마

```
id / name / kind / types[] / languages[] / severity / category / basis(규제 id) / message
kind: forbidden → patterns[] 중 하나라도 매치되면 위반
               → requires_any[] 가 전부 부재하면 위반 (필수고지 누락)
forbidden 추가: condition_absent_any[] - 이 패턴이 본문에 있으면 룰 면제
               suggestion_map          - 매치 문구 → 치환안 템플릿 ({n}=숫자 보존)
required 추가:  fix_append              - [적용] 시 말미에 추가할 고지문
```

4.2 판정 흐름

- 적용 필터:** `content_type ∈ types` 그리고 `language ∈ languages` 인 룰만 평가 (유형③·다국어② 라우팅).
- forbidden:** 모든 패턴의 매치 위치를 `spans[{start,end,text}]` 로 수집 → 에디터 밑줄 하이라이트의 좌표가 됩니다. 면제 조건 예: R-007(금리 표기)은 본문에 `세전|세후` 가 이미 있으면 건너뛴.
- required:** 예) 투자 유형에서 `원금손실이 발생할 수|가능` 계열 문구가 전혀 없으면 R-004 위반. 부정 우회 방지를 위해 "원금 손실 걱정 없이" 같은 문구는 고지로 인정되지 않도록 패턴을 **고지 형태**로 한정했습니다.
- 수정안 생성:** `suggestion_map` 의 `{n}` 템플릿으로 숫자를 보존한 치환안을 만들고("확정 연 5%" → "연 5% (세전, 변동 가능)"), required 룰은 `fix_append` 고지문을 제공합니다. 결과는 `fix:{type: replace|append, ...}` 구조로 F7(수정안 적용)에 그대로 전달됩니다.

4.3 룰 목록 (KO 8종 + EN 2종)

ID	이름	종류	유형	심각도	근거
R-001	단정·확정 표현 (확정 n%, 무조건, 100% 보장...)	forbidden	전체	high	금소법 §22④·영 §20
R-002	원금보장 오인 표현	forbidden	예금·투자	high	표시·광고 심사지침
R-003	근거 없는 최상급 (업계 1위, 최저금리...)	forbidden	전체	medium	표시광고법 §3
R-004	원금손실 고지 누락	required	투자	high	자본시장법 §57
R-005	예금자보호 안내 누락	required	예금	medium	은행연합회 광고심의
R-006	연체이자율·중도상환수수료 누락	required	대출	medium	은행연합회 광고심의
R-007	금리 표기 불충분 (세전·세후 부재 시 n% 표기)	forbidden	예금·대출	medium	심사지침 금리표시
R-008	심사 없는 대출 오인 (무조건 승인, 신용 무관...)	forbidden	대출	high	은행연합회 광고심의
R-101	보장성 오역 표현 — guaranteed, risk-free...	forbidden	전체(EN)	high	내부기준 제2호
R-102	예금자보호 번역 누락 (deposit insurance/KDIC)	required	예금(EN)	medium	내부기준 제2호

5 ④ LLM 심의기 app/pipeline/llm_reviewer.py

5.1 프롬프트 구조

```

system: "한국 금융지주 준법감시인" 역할 + 출력 JSON 스키마 강제
{"issues": [{quote, risk, title, explanation, suggestion, basis_id}], "overall_comment"}
user: [콘텐츠 유형] + [광고 초안]
+ [규제 근거] ← ㉠ RAG 검색 결과 (id·법령·조문 원문)
+ [롤엔진 1차 지적] ← ㉡ 결과
+ 지시: "실제 오인 유발 표현이 있을 때만 롤엔진이 놓친 뉘앙스 리스크를 지적.
        준수 초안이면 빈 배열([]). 본문에 이미 있는 고지(예금자보호 한도·세전·
        기본/우대금리 구분)를 '누락'으로 단정 금지"

```

Ollama 호출은 `format:"json"` + temperature 0.2로 구조화 출력을 강제하고, 파싱 실패 시 1회 재시도합니다
(`llm_client.chat_json`). 세팅값별 근거는 5.4 참조.

5.2 환각 방지 후처리 (핵심)

- **인용 제한:** `basis_id ∉ (검색된 조항 ∪ 룰 근거)` 이면 인용을 제거(null) → LLM이 조항번호를 창작할 수 없음.
- **quote 검증:** `quote` 가 초안 원문에 실제로 존재하지 않으면 비표시 처리 → 없는 문구 지적 방지.
- **중복 제거:** 룰엔진과 (근거, 문구)가 같은 LLM 지적은 리포트에서 제외 → 이중 감점·중복 표시 방지.
- **거짓양성 억제:** 준수 초안이면 `issues=[]` 를 반환하도록 지시하고, 본문에 실재하는 고지를 '누락'으로 단정하지 못하게 프롬프트로 차단 → LLM이 통과 사례를 위반으로 오판하던 환각 제거.
- **참고의견(advisory) 분리:** LLM 단독 지적에는 `advisory:true` 가 붙어 점수 영향에 상한(-6)이 걸리고, 위반·주의 카운트는 룰엔진 기준으로만 집계 → 준법 등급은 결정적 룰엔진이 권위를 갖고 LLM만으로는 등급이 강등되지 않음(7.2 참조).

5.3 폴백

LLM 불가/실패 시 룰엔진 findings를 동일 스키마의 심의 의견으로 결정적 변환하고 `engine: "fallback(rule-based)"` 로 표기합니다.

5.4 LLM 호출 세팅값 — 값·근거

세팅	값	출처	근거
모델	qwen2.5:7b-instruct	<code>OLLAMA_MODEL</code>	Apache 2.0 → 상업적 사용·온프레미스 가능(유료 API 금지 제약 충족) · 한국어 + EN·VI·ZH·JA 다국어 역량(전북 외국인 고객) · 7B는 GPU 8~12GB에서 쾌적. 14B-Q4로 환경변수 1개 교체 가능.
temperature	0.2	고정 상수	준법 심의는 창의성이 아니라 일관성·재현성 이 핵심 — 같은 광고는 같은 판정이어야 감사·신뢰 가능. 완전 결정(0)이 아닌 0.2로 표현 유연성은 최소한 허용하되 판정 흔들림 억제. 운영자가 임의로 못 바꾸도록 환경변수가 아닌 고정.
format	"json"	고정 상수	출력을 JSON 스키마로 강제 → 파싱 안정성·파이프라인 통합, 자연어 잡담 차단.
retries	1	고정 상수	JSON 파싱 실패는 드물어 1회 재시도로 회복 충분. 무한 재시도는 지연·자원 낭비 → 1회 실패 시 즉시 룰엔진 폴백이 더 안전.
호출 타임아웃	120초	<code>LLM_TIMEOUT</code>	7B는 CPU 추론 시 수십 초 가능 → 넉넉히 120초. GPU면 훨씬 빠름. 초과 시 폴백으로 강등.
헬스체크 타임아웃	2.0초	고정 상수	Ollama 생존 확인(<code>/api/tags</code>)은 즉답이어야 UX가 안 막힘 → 짧게. 미응답이면 폴백 모드로 판단.

6 ⑥ 소비자 오인 시뮬레이터 `app/pipeline/simulator.py` · `app/data/personas.json`

헤드라인 기능. **2단계 구조**: (1) 페르소나 역할극 — 광고를 1인칭으로 읽기, (2) 오해 판정기(judge) — '이해'를 사실과 대조해 오해만 추출. 페르소나 3종은 병렬(`asyncio.gather`)로 추론됩니다.

6.1 페르소나 정의

페르소나	프로필 요지	취약 카테고리
 72세 은퇴자	금융이해도 낮음, 안전 최우선, '보장·안심'에 반응, 세부조건 안 읽음	guarantee, assertive
 20대 금융초보	용어 약함, 단기수익 기대, 숫자(%)에 끌리고 작은 글씨 건너뛸	assertive, rate, superlative
 외국인 거주자	한국어 중급, 금융용어·예금자보호 제도 모름 (전북은행 고객 특성)	notice, rate, guarantee

6.2 LLM 모드 — 역할극 → 판정 2회 호출

1차 (역할극): `system = "당신은 아래 프로필의 실제 금융소비자, 전문가 아님"`
`user = 프로필 + 광고 → {understanding(1인칭 이해), action(행동), expectations[]}`

2차 (판정기): `system = "금융소비자보호 심의역 - 이해를 ㉔상품 실제 조건,
㉔규제상 암시 금지 항목(원금보장·확정수익·무심사)과 대조"`
`user = 프로필 + 1차 결과 + 광고 원문 + product_facts`
`→ {misunderstandings:[{text, risk, trigger_phrase, category}]}`

- **유발 문구 역추적**: 판정기가 반환한 `trigger_phrase` 가 광고 원문에 실제 존재하는지 검증, 없으면 폐기.
- **근거 연결**: `category → 조항` 고정 매핑 (guarantee→금소법 §22㉔, assertive→§22㉔, rate→심사지침, notice→§22㉔, superlative→표시광고법 §3) — 차별 기능이 본 심의의 설명가능성을 강화하는 연결 고리.
- 역할극 출력이 스키마에 어긋나면 해당 페르소나만 폴백으로 강등.

6.3 폴백 모드 — 룰 기반 결정적 시뮬레이션

`hit = {㉔ 룰엔진 findings의 category 집합}`
`key = 페르소나 취약 카테고리 중 hit에 포함된 첫 항목 (없으면 hit n 보유 템플릿, 그것도 없으면 "safe")`
`출력 = personas.json의 해당 카테고리 폴백 템플릿(이해/행동/오해/위험도)`
`trigger_phrase = 같은 카테고리 룰의 matched_text (required 룰이면 "(고지 문구 자체가 없음)")`

6.4 종합 오인 리스크

`high 오해 ≥ 2건 → "높음" · 오해 ≥ 1건 → "중간" · 없음 → "낮음"`

7 ㉔ 리포트 종합 — 준법점수 산식 `app/pipeline/report.py`

7.1 감점표

`점수 = 100`
`- ∑ 룰엔진(결정적 권위): high -12 · medium -4 · low -1`
`- min( ∑ LLM 고유 지적(룰 중복 제외): high -5 · medium -2 · low -1 , 상한 6 ) ← 참고의견 영향 제한`
`- min( ∑ 시뮬레이터 오해: high -4 · medium -2 , 상한 12 )`
`하한 5점`

LLM·시뮬레이터 감점에 각각 상한(-6/-12)을 둔 근거: 이들은 비결정적 보조 신호이므로 점수를 흔들 수는 있어도 **지배하지는 못하게** 제한. 룰엔진 감점만 상한이 없습니다(결정적 권위).

7.2 등급 판정

룰 위반 0건(결정적으로 준법)이면 LLM·시뮬레이터 참고의견만으로 위험 강등 불가:
룰 감점 0 → 점수 ≥ 80 → ● 통과, 그 외 → ● 주의

룰 위반이 있으면:

- 통과(pass) : 점수 ≥ 85 그리고 룰 high 0건
- 위험(danger) : 점수 < 60 또는 (룰 high ≥ 1건 그리고 점수 < 75)
- 주의(caution) : 그 외

등급 게이팅의 **high**는 **룰엔진 high만** 셉니다(LLM 단독 high는 advisory로 제외). 두 가지 비대칭 규칙: ① 룰 high 1건이면 점수가 다소 높아도 위험으로 끌어내려 "치명적 위반 1건"의 통과 위장을 방지하고, ② 룰이 깨끗하면(결정적 준법) LLM 환각이 통과 사례를 위험으로 오판하지 못하게 보호합니다. 후자는 qwen2.5-7B가 본문에 존재하는 고지를 '누락'으로 단정 하던 거짓양성(통과 사례 81→주의)을 88·통과로 바로잡은 근거입니다.

7.3 하이라이트 병합

- 룰 span + (원문에 존재 검증된) LLM quote 위치를 모아 시작 위치순 정렬.
- 프론트는 겹치는 구간을 앞선 span 우선으로 제거 후, 투명 textarea 뒤의 backdrop 레이어에 **<mark>** (빨강=high, 노랑=medium)로 렌더링 — 에디터 자동 밀줄의 원리.
- 모든 지적·오해의 **basis_id**는 코퍼스 메타데이터(법령·조항·출처 URL)로 해석되어 인용 카드로 표시됩니다.

8 F7 수정안 적용 orchestrator.apply_fixes

replace 타입: text.replace(original → replacement) # 동일 문구 전체 치환
append 타입: 고지문이 본문에 아직 없으면 말미에 줄바꿈 후 추가 (중복 방지)
적용 후 프론트가 자동 재심의 → before/after 점수 변화 확인

8.1 준법 오토파일럿 (F12) orchestrator.autopilot · remediator.py

F7를 사람이 1회 누르는 대신, 에이전트가 **심의→고쳐쓰기→재심의**를 통과/종료조건까지 자율 반복한다. 합격 판정은 결정적 룰엔진(**run_review** 의 grade)만 — LLM이 통과를 위장할 수 없다.

```
for i in range(MAX_ITER = 4):
    report = run_review(current) # 판단 (기존 파이프라인 재사용)
    if report.grade == "pass": return ✅ 수렴 # 종료조건
    if score < prev: return ■ regressed # 발산 방지(직전 유지)
    if score == prev: return ■ plateau # 정체

    fixed = apply_fixes(current, 룰 치환·고지) # 행동 (a) 결정적 우선
    residual= 룰 fix 없는 위반 + LLM 참고의견
    current = remediator.remediate(fixed, residual) # 행동 (b) 잔여만 LLM 재작성
    if current 불변: return ■ no_change

trace = [{iter, draft, report, score, grade, changes[{before,after,basis_id,engine}]}]
```

- 하이브리드 수정**: 정형 위반은 결정적 룰 치환(무료·재현가능), 비정형 위반(최상급 등)만 LLM 재작성 → 비용·지연·환각 표면적 최소화.
- 수정 에이전트**(**remediator.py**): "마케팅 효과(핵심 셀링포인트) 보존 + 새 위반·허위고지 금지" 제약 프롬프트. 인용은 검색된 조항 화이트리스트로 한정. LLM 불가 시 원문 유지(폴백) → 루프는 '미수렴 → 사람 에스컬레이션'으로 종료.

- 입력 이벤트마다 타이머 리셋 → **1.2초 멈춤** 시 심의 호출 (디바운스). 붙여넣기는 0.35초로 단축.
- 유형/언어 변경 시 즉시 재심의. 상태 표시등: 회색(대기) → 노랑 점멸(심의 중) → 초록(완료).
- 서버측 가드: 8자 미만은 `skipped=true` 로 즉시 반환 (LLM 호출 비용 차단).

12 에러 처리 매트릭스 (설계 \$15 대응)

리스크	구현된 방어	위치
LLM 출력 파싱 실패	<code>format:"json"</code> 강제 + 코드펜스/잡담 제거 파서 + 1회 재시도 → 실패 시 룰 기반 폴백	llm_client / llm_reviewer
RAG 무관 조항 검색	top-k=4 + 임계값 0.05 + "근거 불충분" 표기	retriever
LLM 환각 인용	basis_id 화이트리스트(검색 코퍼스 한정) + quote 원문 존재 검증	llm_reviewer / simulator
빈/초단문 입력	8자 미만 검사 스킵 + 안내	orchestrator
번역 오역·고지 누락	영문 룰셋 재심의 (R-101/R-102)	multilingual + rules
Ollama 다운(런타임)	호출 예외 → 해당 모듈만 폴백 강등, 파이프라인은 완주	전 모듈
오토파일럿 무한루프·발산	MAX_ITER 상한 + plateau/no_change/regressed 조기 종료 + 합격은 룰 엔진만 판정	orchestrator.autopilot

13 테스트 전략 tests/ · app/data/fixtures.json

- **TDD 정답셋**: 픽스처 5종(위반 4 + 통과 1)에 `expected_rule_ids / expected_type / expected_grade` 를 명시 → 룰·분류·등급 회귀 테스트가 파라미터라이즈로 자동 생성됩니다. 픽스처 = 데모 입력 = 정답셋 (삼중 활용).
- **결정성**: 통합 테스트는 `llm_client.is_available` 을 monkeypatch로 False 고정 → 폴백 모드에서 항상 동일 결과.
- **시나리오 검증**: 수정안 적용 → 점수 +20 이상 상승 & high 0건 / 72세 페르소나의 "원금 보장" 유발 문구 검출 / 직역 'guaranteed' → R-101·R-102 검출 / 인용은 코퍼스 내 조항만.
- **오토파일럿**(`test_autopilot.py`): 점수 단조 개선 · 종료조건 보장(무한루프 방지) · 합격 무결성(룰엔진 grade==pass일 때만 수렴) · 폴백 모드 동작. **총 38개 테스트 통과.**

14 알려진 한계

- 규제 코퍼스는 큐레이션 서브셋(12조항)이며 조문 원문을 요약·재구성한 것 → 운영 적용 시 전문 수록 및 법무 검증 필요.
- 룰엔진 정규식은 표기 변형(띄어쓰기·이형태)에 대한 커버리지 한계가 있음 → 형태소 분석 기반 매칭으로 고도화 가능.
- 폴백 시뮬레이션은 템플릿 기반이라 페르소나 발화가 고정적 → LLM 모드에서 본래 성능 발휘.
- 준법점수 가중치는 데모 기준 휴리스틱 → 실제 심의 데이터로 캘리브레이션 필요.

JB 준법 코파일럿 로직 설명서 v0.1.1 · 2026-06-13 · 소스: app/pipeline/* · 데이터: app/data/* · 변경: 준법 오토파일럿(8.1) 추가, 세팅값 근거표 (3.4·5.4), LLM 거짓양성 억제·advisory 분리(5.2·7) 반영